SCGSR Final Report Additional Materials

Jacob Hauck

August 30, 2026

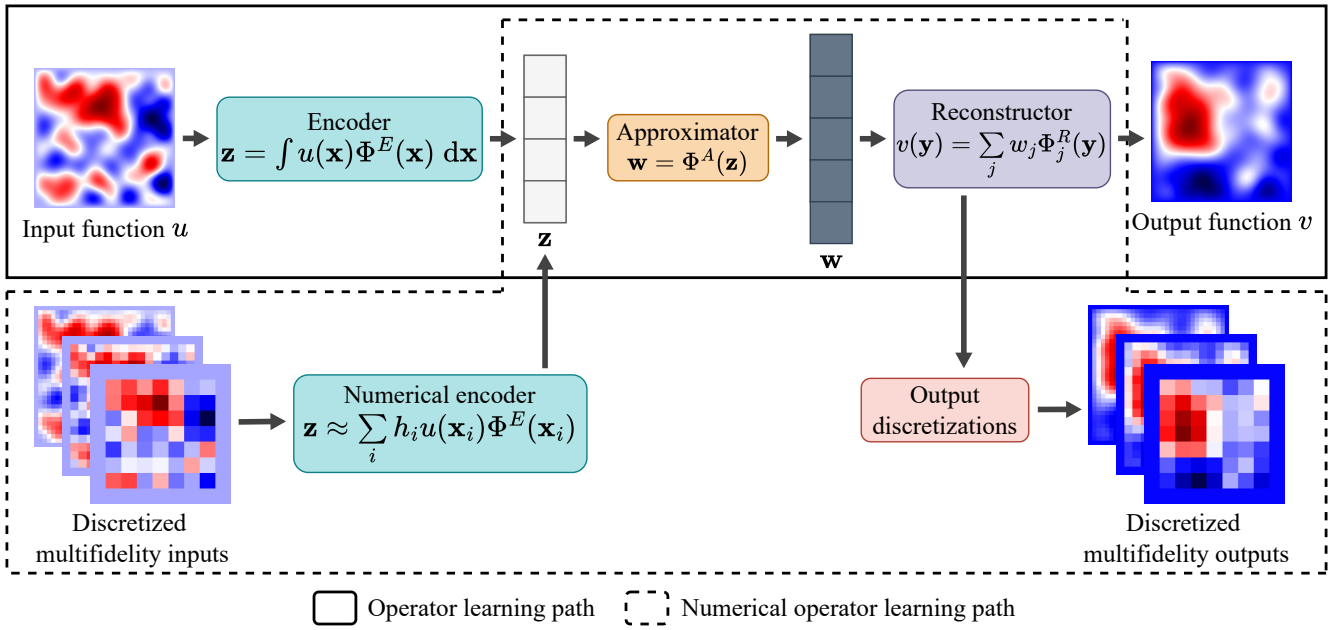


Figure 1: The architecture of our operator learning method, and how operator learning enables discretization independence through numerical operator learning. Note the three-module structure: (linear) encoder projects the input function onto a finite-dimensional space parameterized by neural network basis functions (Φ^E), approximator approximates the operator in the latent space, and (linear) reconstructor builds the output function by taking a linear combination of neural network basis functions (using a different neural network Φ^R) using the latent output variables from the approximator as weights. Vector-valued extension is described in (1).

$$\begin{cases} E(\theta_E; u) = \left(\langle u, \Phi_1^E\rangle, \dots, \langle u, \Phi_p^E\rangle\right) \in \mathbb{R}^p, & u \in \mathcal{U} = L^2(\Omega_{\mathcal{U}}; \mathbb{R}^{c_{\mathcal{U}}}), \\ A(\theta_A; \mathbf{z}) = \Phi^A(\mathbf{z}) \in \mathbb{R}^q, & \mathbf{z} \in \mathbb{R}^p, \\ R(\theta_R; \mathbf{w}) = \sum_{j=1}^q w_j \Phi_j^R \in \mathcal{V} = L^2(\Omega_{\mathcal{V}}; \mathbb{R}^{c_{\mathcal{V}}}), & \mathbf{w} \in \mathbb{R}^q, \end{cases}$$

where

$$\Phi^E(\mathbf{x}) = \begin{bmatrix} \varphi_1^E(\mathbf{x}) \\ \varphi_2^E(\mathbf{x}) \\ \vdots \\ \varphi_p^E(\mathbf{x}) \end{bmatrix}, \quad \mathbf{x} \in \Omega_{\mathcal{U}}, \qquad \Phi^R(\mathbf{y}) = \begin{bmatrix} \varphi_1^R(\mathbf{y}) \\ \varphi_2^R(\mathbf{y}) \\ \vdots \\ \varphi_q^R(\mathbf{y}) \end{bmatrix}, \quad \mathbf{y} \in \Omega_{\mathcal{V}},$$

$$\varphi_i^E : \Omega_{\mathcal{U}} \to \mathbb{R}^{c_{\mathcal{U}}}, \qquad \varphi_i^R : \Omega_{\mathcal{V}} \to \mathbb{R}^{c_{\mathcal{V}}}. \tag{1}$$

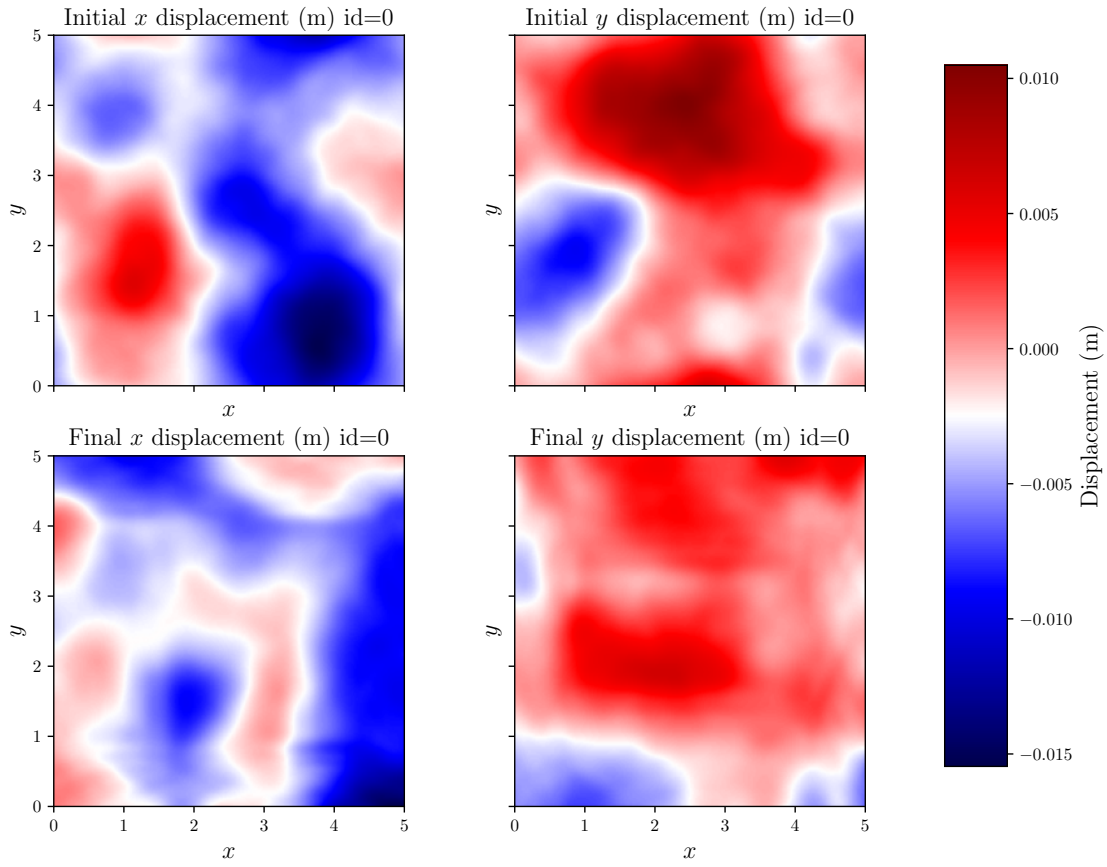


Figure 2: Example input–output pair from one of the wave propagation benchmark datasets, prepared using `PDMATLAB2D`. The input function is the top row, which shows the initial x and y displacement fields. The output function is the bottom row, which shows the final x and y displacement fields after a time $T = 1.8$ seconds has elapsed in the simulation. The "id=0" refers to this sample's ID in the dataset.

Approximator	Error on training data	Error on testing data
Base (3 layers, 128 neurons/layer, ReLU)	15.2	18.1
Wide (3 layers, 384 neurons/layer, ReLU)	12.4	20.7
Deep (8 layers, 128 neurons/layer, SELU)	11.9	16.9

Table 1: Error (measured in % relative L^2) for different approximator network settings. Using a wide approximator leads to overfitting (note gap between train and test error in the second row), but using a deep approximator with self-normalizing activation leads to a slight performance boost (compare error in first and third rows).

γ	Error on training data	Error on testing data
2.5	14.3	18.6
3.1	7.6	10.7
4.0	4.6	7.1
5.0	4.0	5.7

Table 2: Error (measured in % relative L^2) for different input distribution complexities, as measured by the γ decay rate of the covariance eigenvalues; smaller γ means more complex. We observe that the more complex data is more difficult for operator learning models to fit, as measured both on the training and testing data.

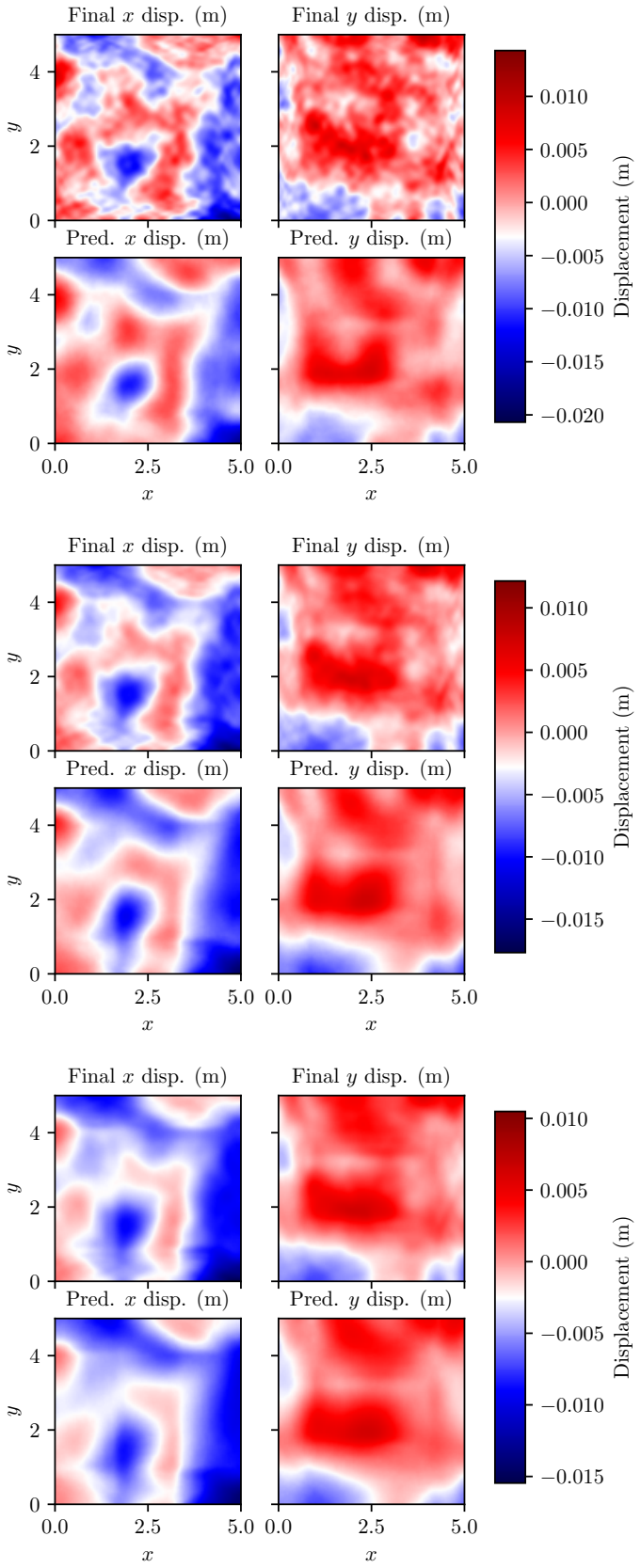
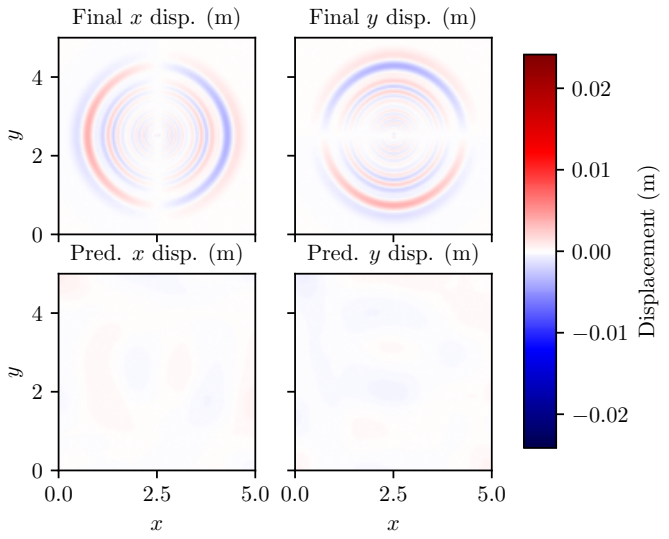
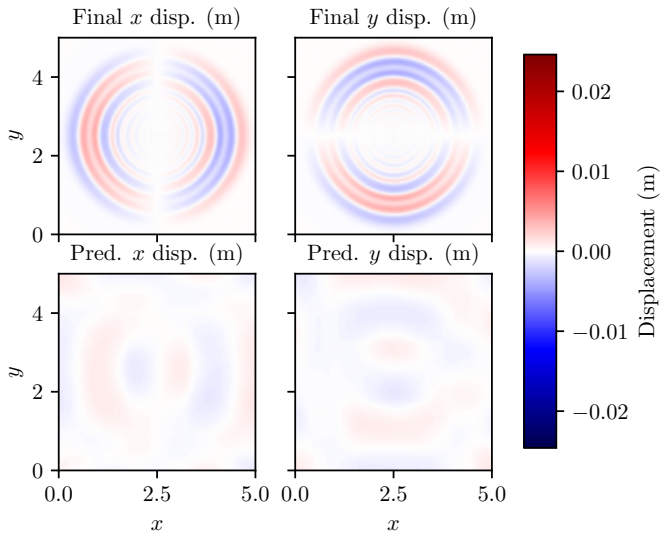


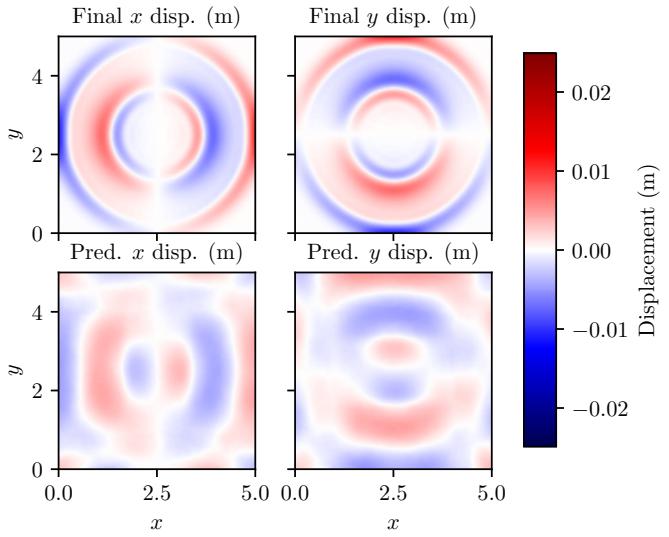
Figure 3: Example final displacement fields (top rows, x and y components alternating) and corresponding model predictions (bottom rows) for different input Gaussian process covariance eigenvalue decay rates. Decay rate increases from top to bottom. Note the corresponding decrease in strong high-frequency components from top to bottom. Also note how the model prediction only capture low-frequency (smooth) components, making its predictions more accurate when the covariance eigenvalues decay quickly.



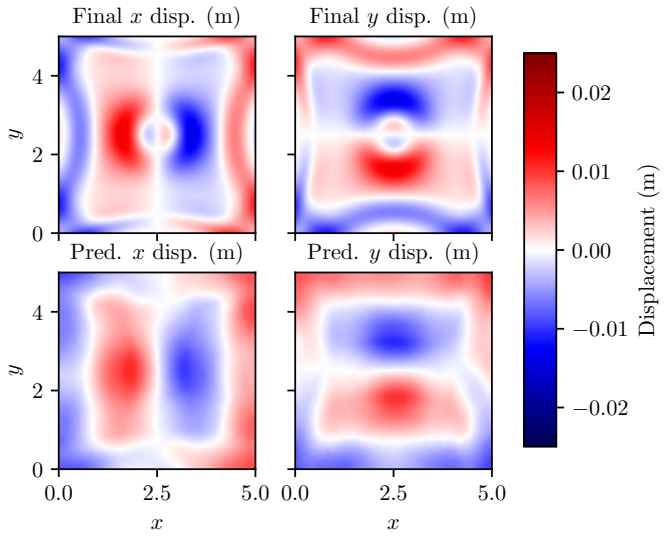
(a) Initial displacement size = 0.03



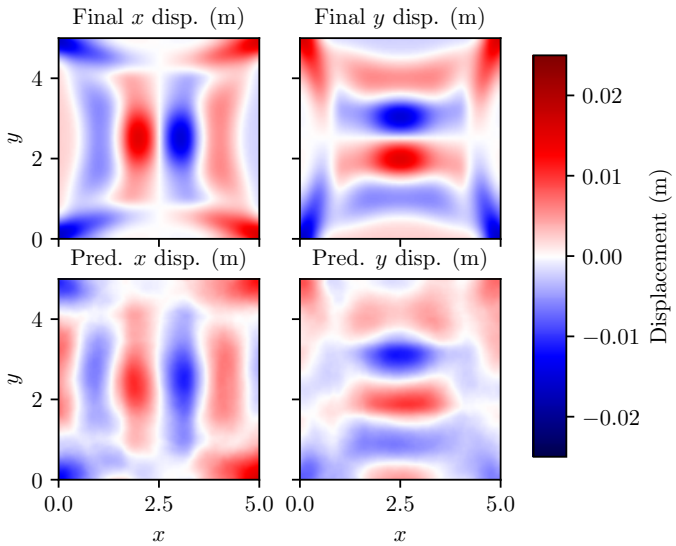
(b) Initial displacement size = 0.05



(c) Initial displacement size = 0.10



(d) Initial displacement size = 0.20

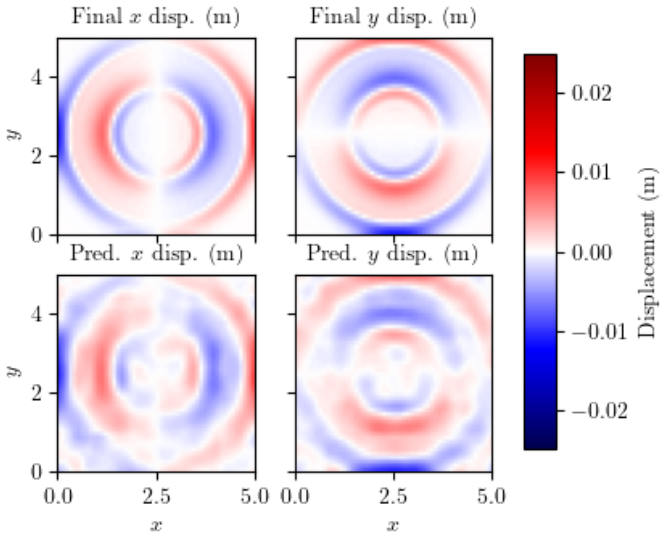


(e) Initial displacement size = 0.30

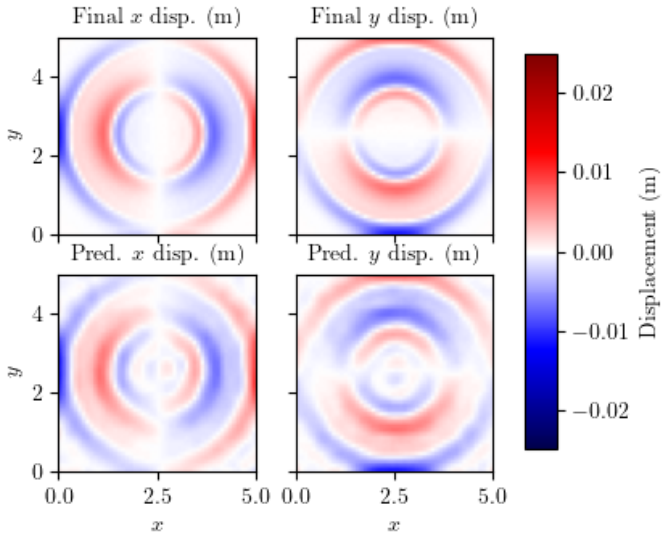
Figure 4: Model predictions for out-of-distribution initial displacements. The top rows in each block show simulation final displacement field x and y components, and the bottom rows show the corresponding model predictions. Input initial displacement fields are Gaussian bumps of varying sizes, specified in the captions of each subfigure. The model makes more accurate predictions for larger bumps (which have less high-frequency information and result in less dispersive effects).

T	Error on training data	Error on testing data
1.8	15.4	18.2
1.4	16.5	21.0
1.0	16.0	19.3
0.6	14.1	17.1
0.2	14.4	19.2

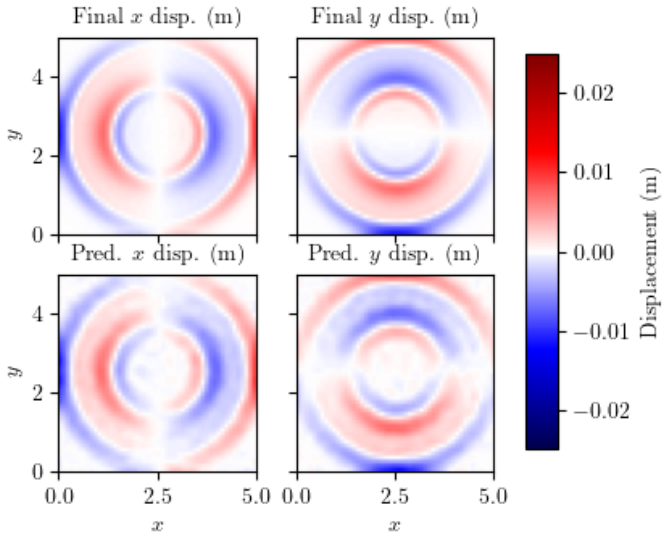
Table 3: Independence of model error (measured in % relative L^2) on simulation duration T. When $T = 0.2$, the operator is nearly the identity, suggesting that the reconstructor module in our model is insufficient. As illustrated in Figure 5, the error of our model can be reduced by increasing the number of basis functions in the encoder and reconstructor.



(a) 256 basis functions

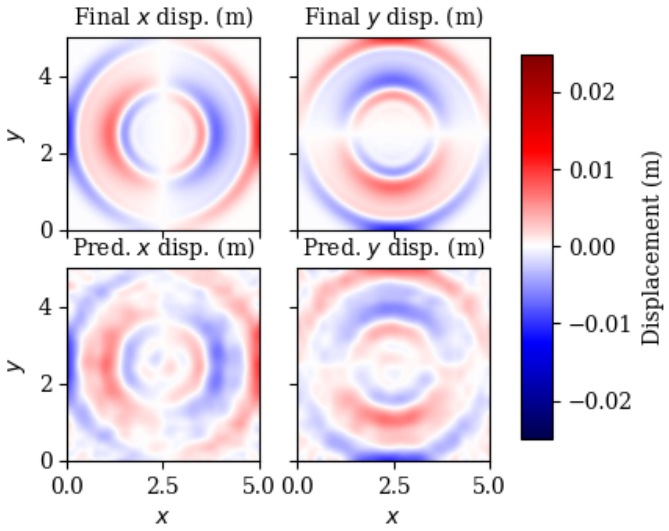


(b) 512 basis functions

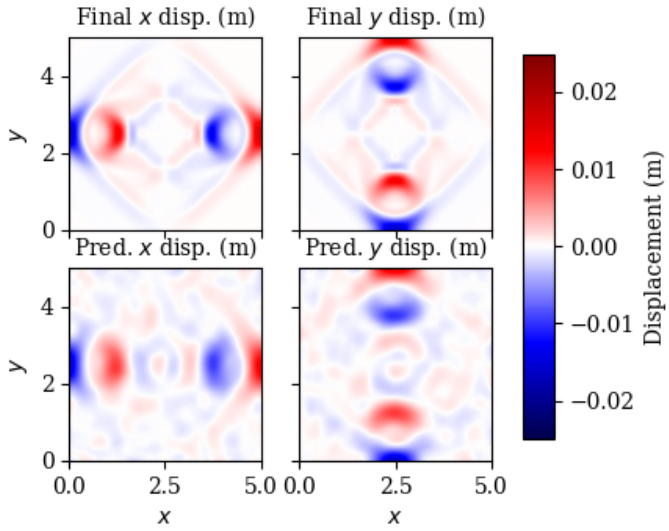


(c) 1024 basis functions

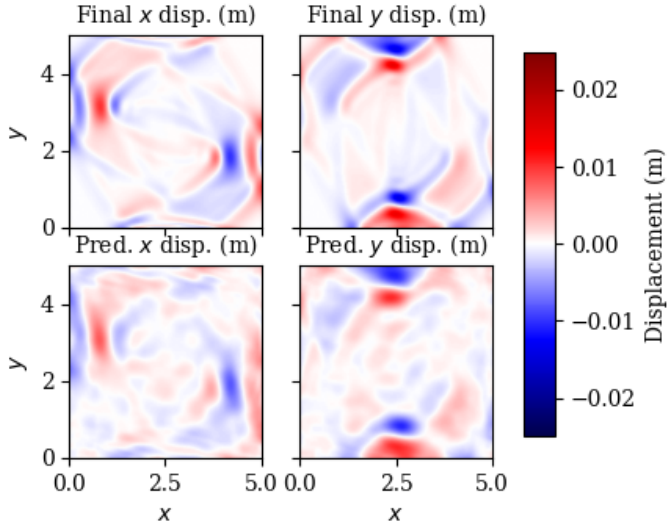
Figure 5: Model predictions for out-of-distribution for initial displacements for different model sizes compared to true final states (top rows), as measured by the number of basis functions in the encoder/reconstructor modules (p and q in (1)). We need to increase the number of basis functions substantially in order to achieve good accuracy, which is a symptom of the challenges of hyperbolic problems (such as elasticity) for operator learning. Note that even fewer basis functions (128) were used in Figure 4, which explains why the accuracy there is even lower than in any of the above cases.



(a) Isotropic symmetry



(b) Square symmetry



(c) Oblique symmetry

Figure 6: Final conditions under different material symmetry models resulting from the same initial condition, along with operator learning predictions from models trained on datasets with corresponding symmetry. Top rows: true final states. Bottom rows: operator learning predictions. We observe similar accuracy across all symmetry types.

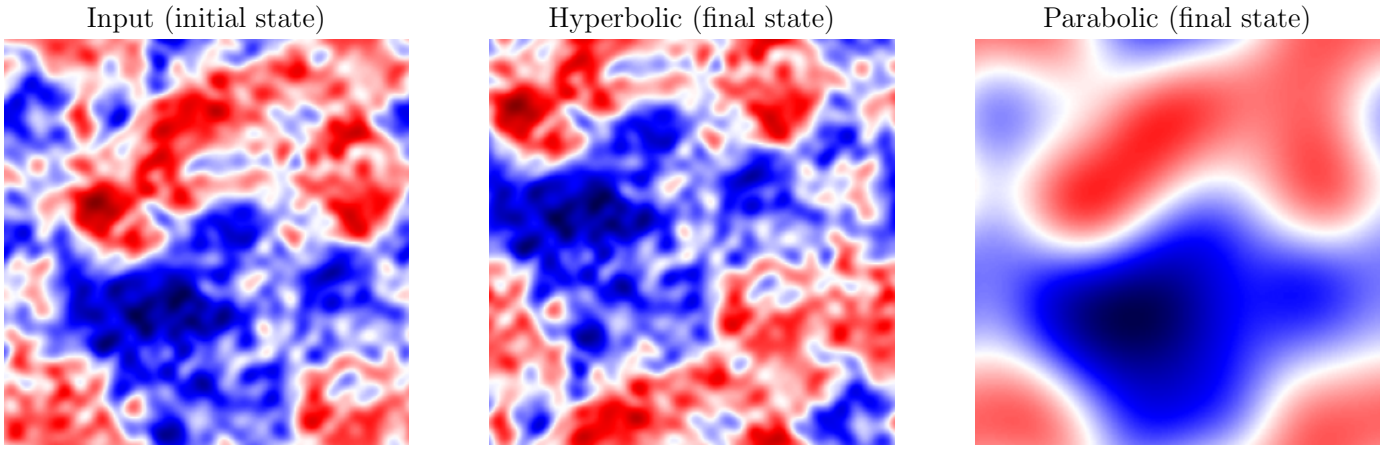


Figure 7: Comparison of propagation operators for hyperbolic and parabolic systems. The final state in the hyperbolic problem retains complex (high-frequency) features in the initial state whereas the final state in the parabolic problem is significantly simpler, with all the fine details smoothed out.

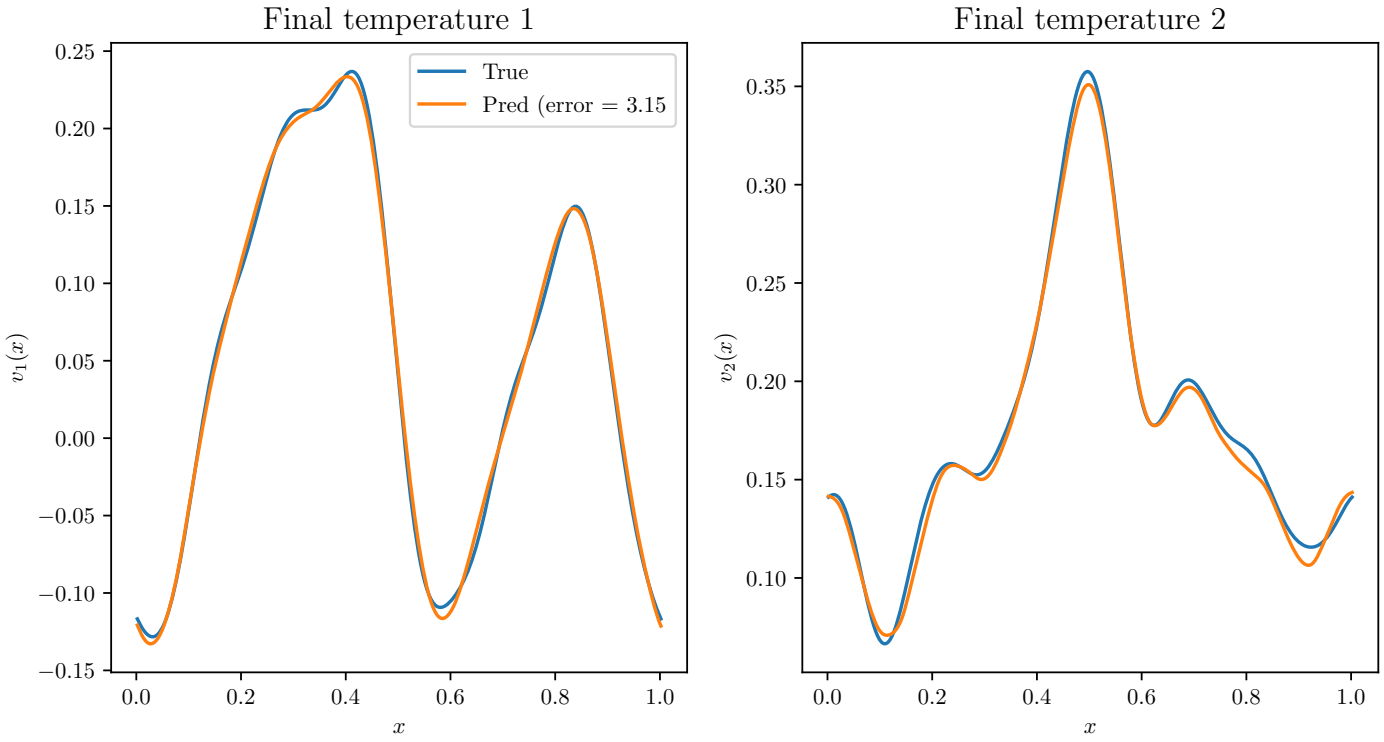


Figure 8: Example final state in the 1D heat equation for two samples (labeled temperature 1 and temperature 2 above, as the state in the heat equation is a temperature field rather than a displacement field). We can see that the final states are smoother than in the wave case in Figure 9, and the operator learning model makes quite accurate predictions (3.15% relative L^2 error).

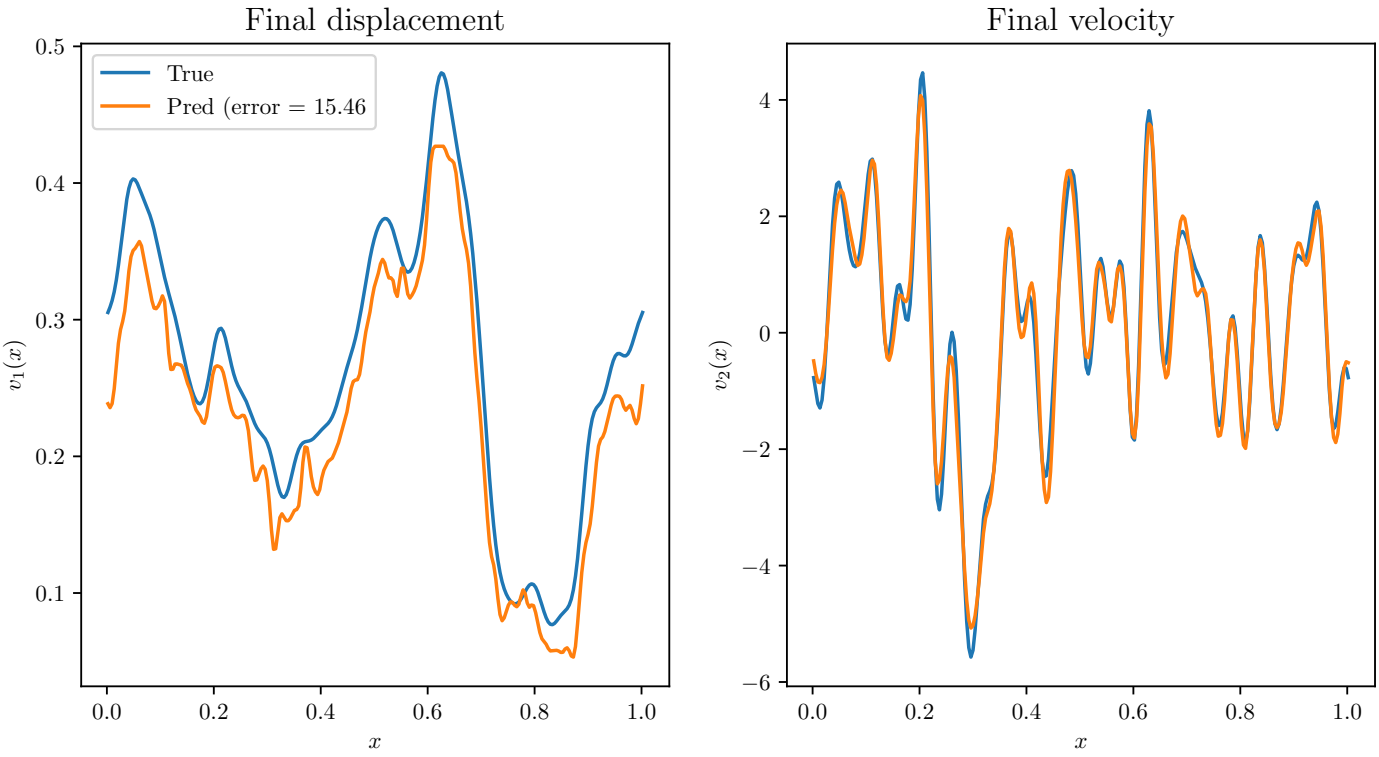
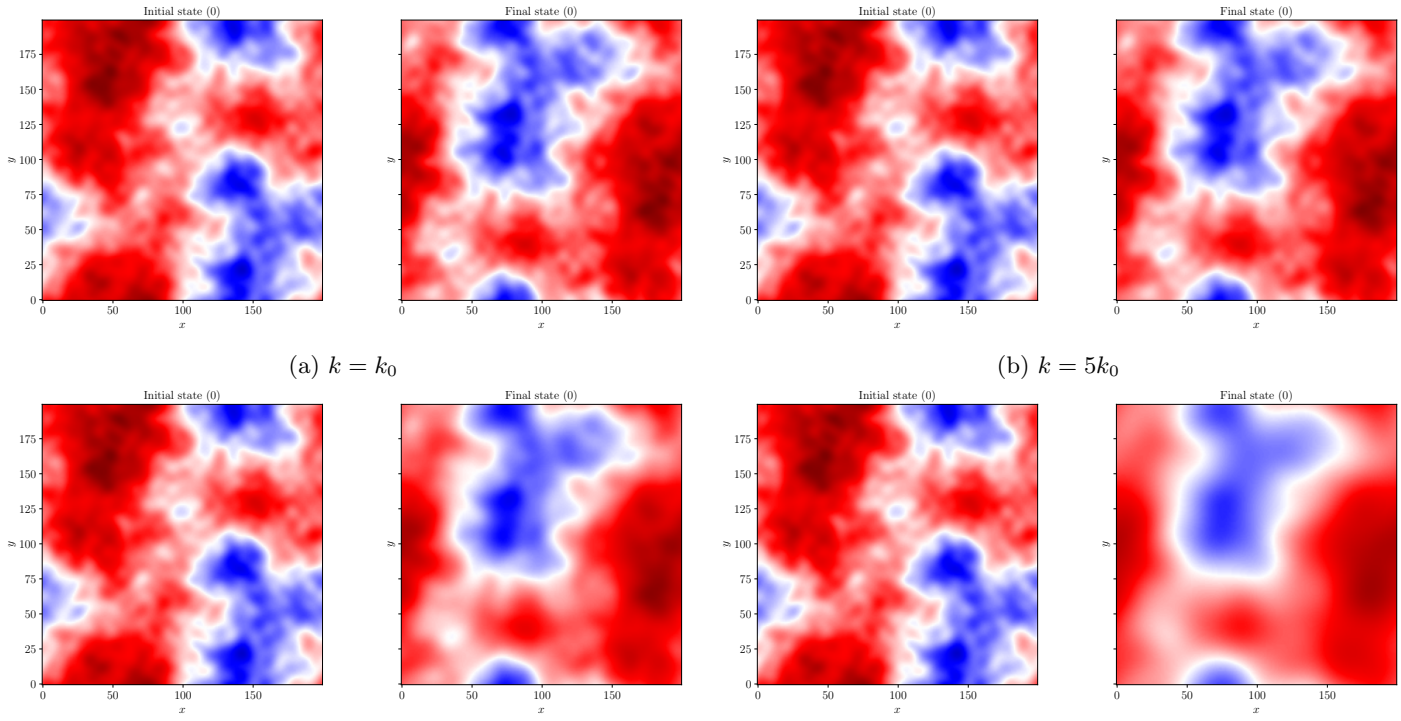


Figure 9: Example final state for the 1D wave equation (here we predict both the final displacement field and velocity field as a vector-valued target), using the same initial state as in the heat equation in Figure 8—we use dimensionless units in both equations to allow direct comparison. We can see that the final states here are more oscillatory than in the heat equation in Figure 8, and the operator learning model makes less accurate predictions (15.46% relative L^2 error).



(a) $k = k_0$ (b) $k = 5k_0$

(c) $k = 25k_0$ (d) $k = 125k_0$

Figure 10: Sample input and output functions for the 2D advection–diffusion equation with different values of the diffusion constant k. Since the same seed was used to generate each dataset, the initial state (left function in (a–d)) is the same in all cases, but the final state (right function in (a–d)) becomes smoother as k increases because the physics become more diffusion-dominated. We expect to use these datasets to show that operator learning consequently has higher performance for higher values of k, that is, when the hyperbolic behavior of the advection term is more dominated by the parabolic behavior of the diffusion term.

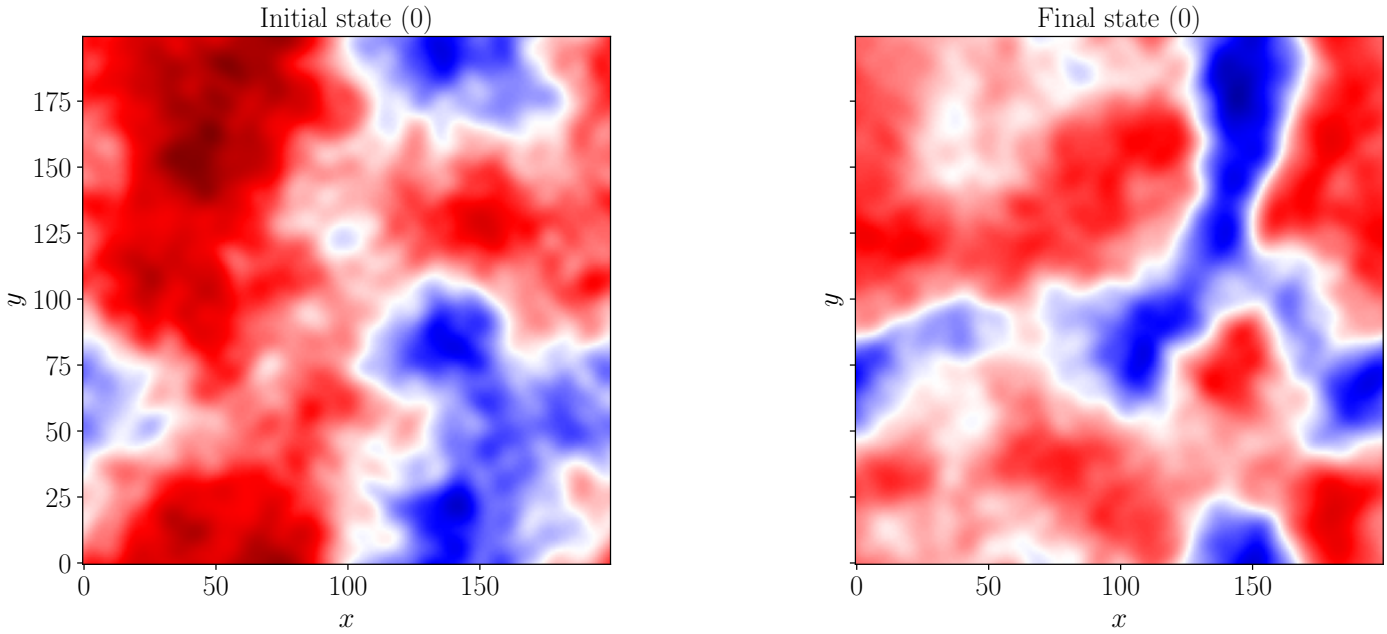


Figure 11: Sample initial and final states from the nonlinear wave equation dataset. For planned operator learning experiments, the input is the left function and the output is the right function.

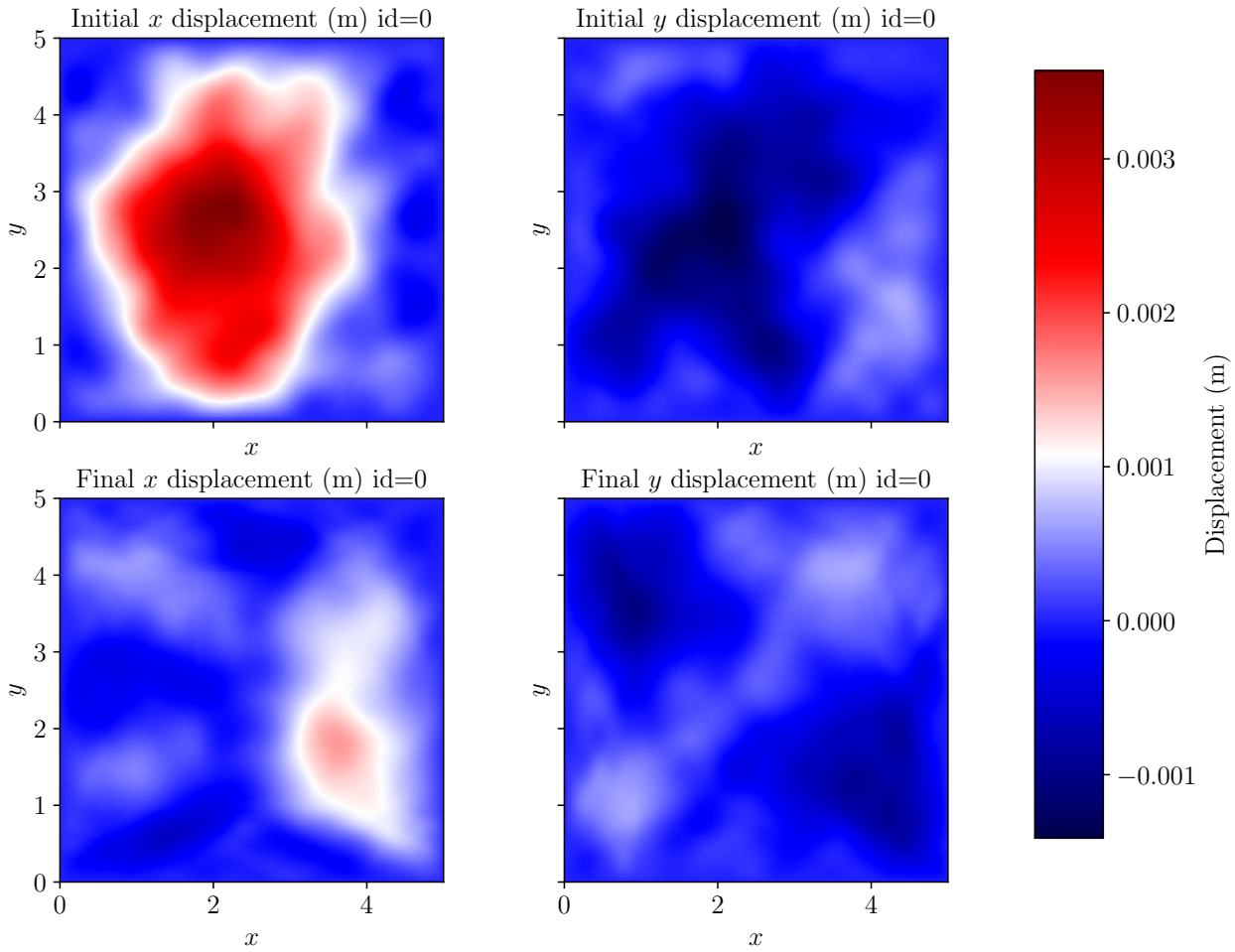
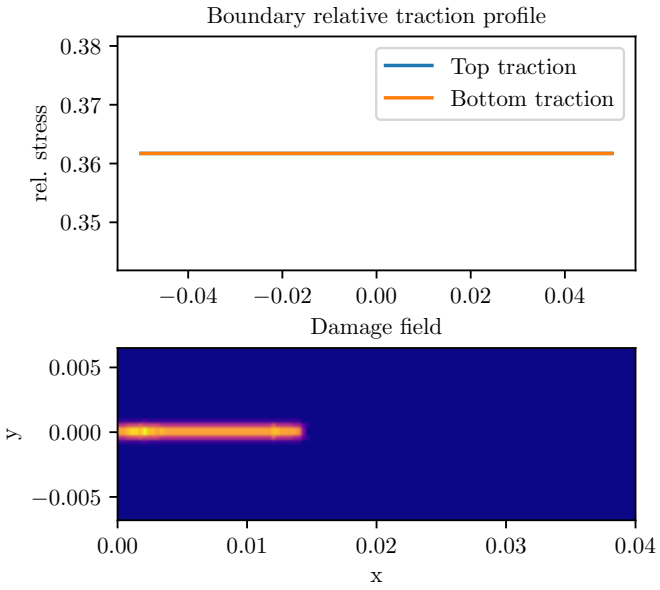
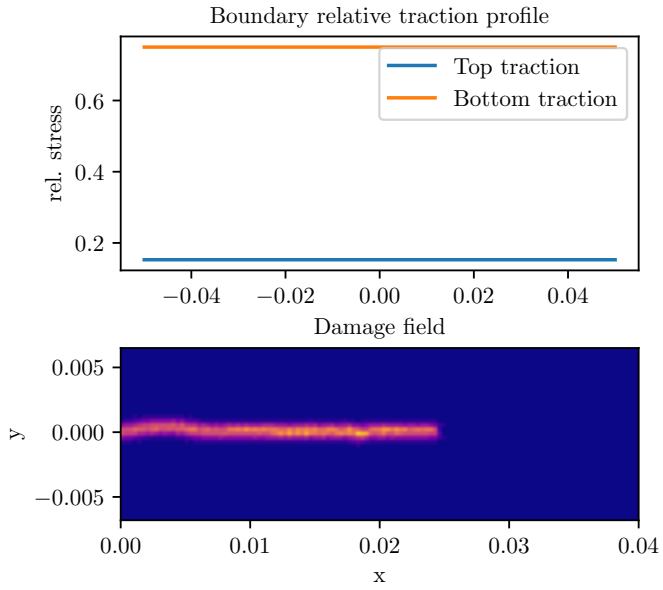


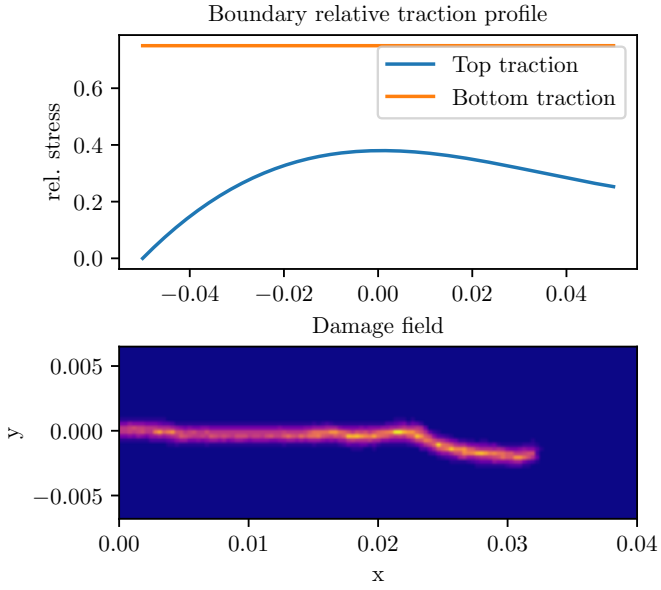
Figure 12: Sample initial and final states from the classical elasticity dataset. For planned operator learning experiments, the input is the top row and the output is the bottom row; the left column is the x displacement, and the right column is the y displacement.



(a) Symmetric, uniform traction



(b) Asymmetric, uniform traction



(c) Asymmetric, uniform traction

Figure 13: Samples from generated crack propagation datasets. The input function is the boundary relative traction profile (top row, units are relative to a fixed, maximum traction), and the output function is the final damage field (bottom row, cropped to the region around where the crack grew). Note that dark blue means no damage, and orange/white means high damage. The goal for operator learning on these datasets is to predict the final crack path.

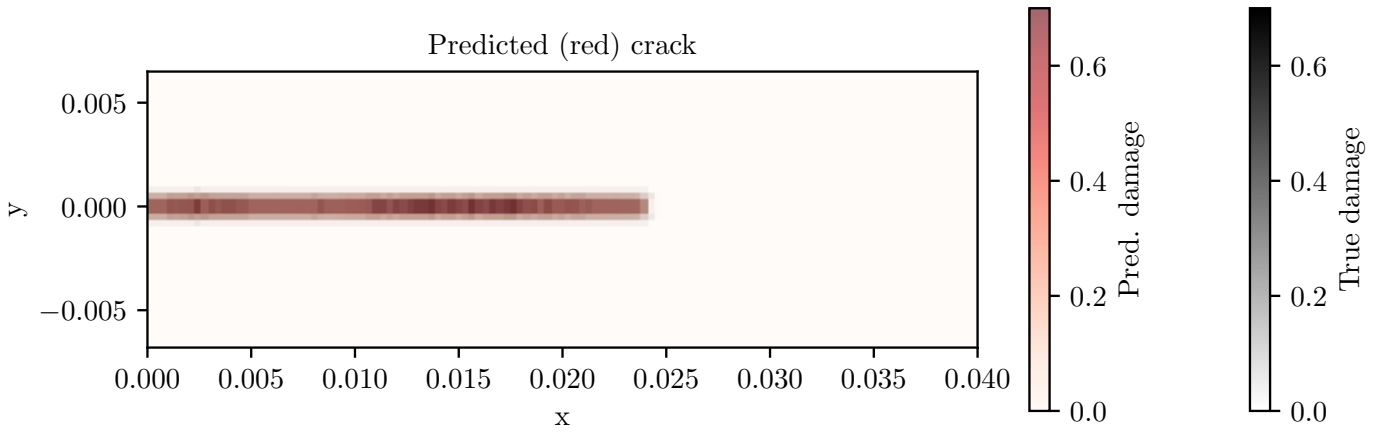


Figure 14: An example crack final damage field (black) and model prediction superimposed (red) in the case of symmetric traction on the top and bottom of the plate. This image is cropped to show only the part of the crack that grew. Note that the damage field refers to the fraction of broken bonds (between 0 and 1) per particle. The model prediction agrees closely with the true crack path.

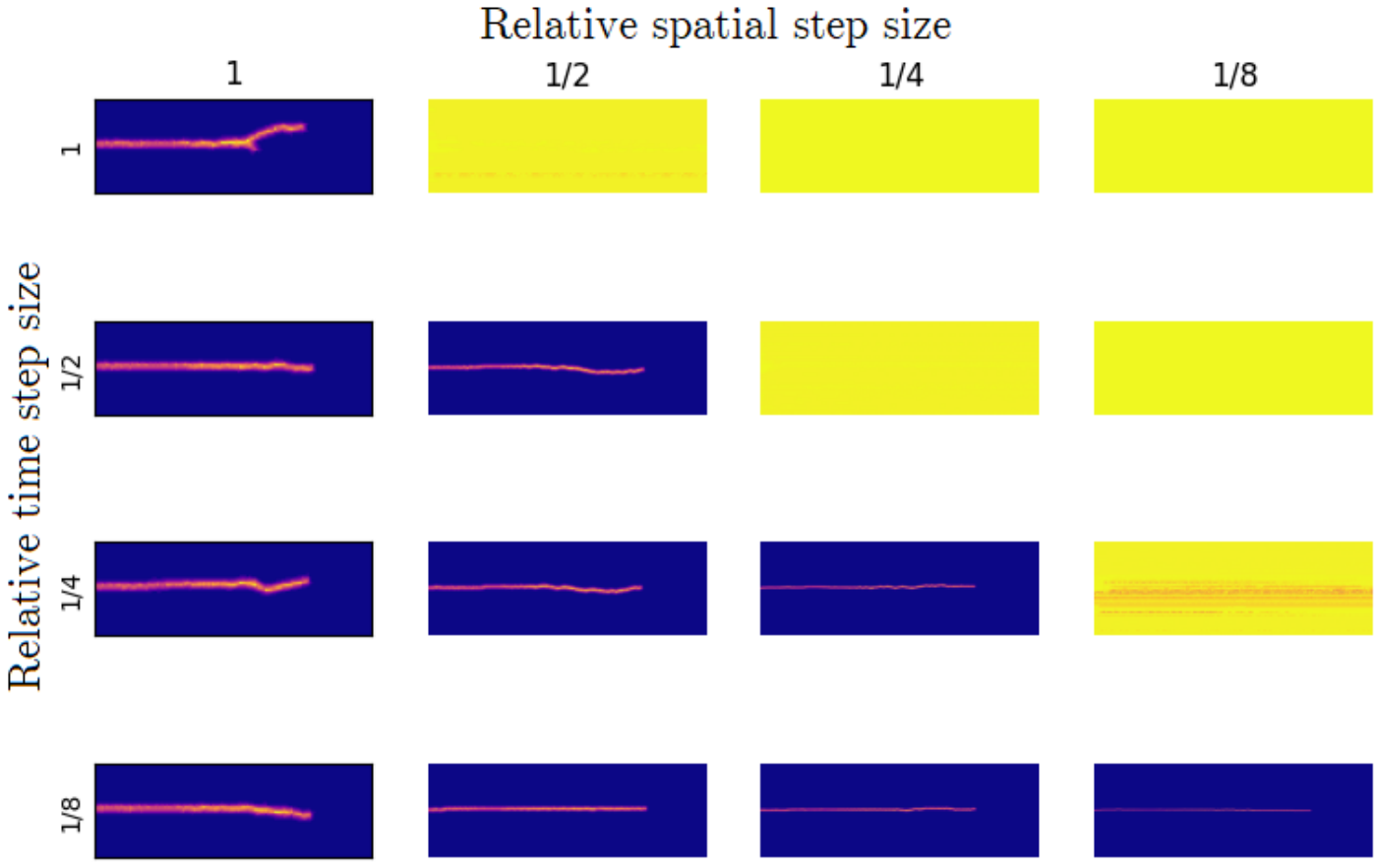


Figure 15: Study of numerical convergence of crack propagation with asymmetric, constant loading with respect to discretization parameters. Each column corresponds to a fixed grid spacing Δx and each row corresponds to a fixed time step size Δt in the numerical simulation. The same peridynamic horizon δ is used in all examples. Yellow indicates numerically unstable cases (caused by the time step size being above the maximum stable time step size for the grid spacing Δx). We expect convergence to a common solution as Δx and Δt decrease, but the results highlight the solution's sensitivity to the discretization parameters.

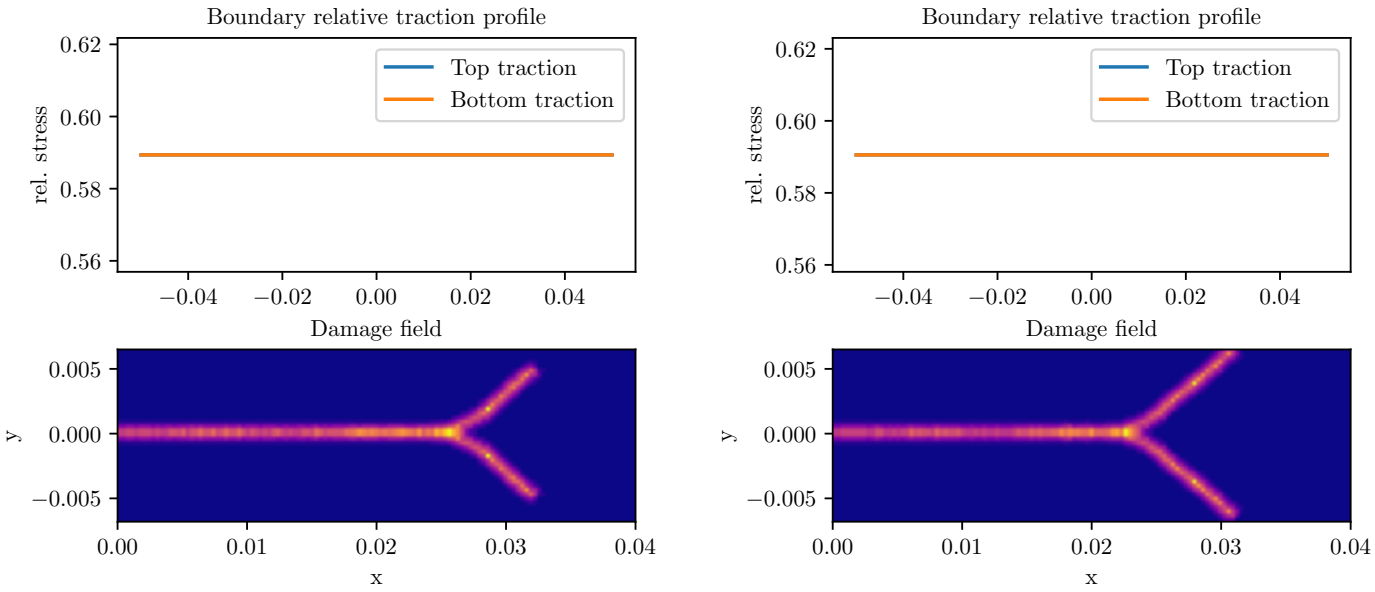


Figure 16: Comparison of simulated crack paths with symmetric, uniform relative traction that differs in strength by less than 1%, as depicted in the top two plots, which are nearly indistinguishable as a result. The simulation produces very different results despite the nearly identical parameters. This makes prediction with operator learning quite challenging, so identifying a stable configuration will be a top priority for future work following my internship.

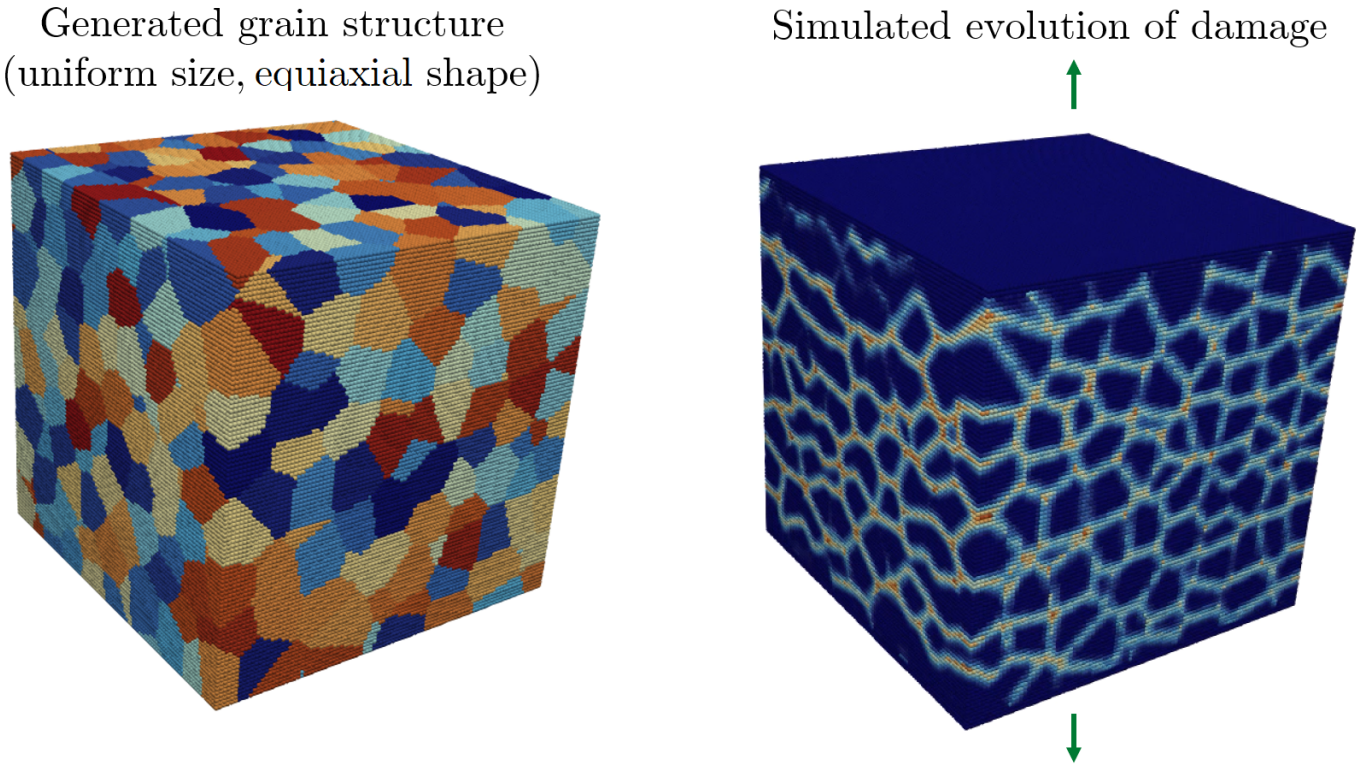


Figure 17: Left: example grain structure from my custom Voronoi tessellation algorithm that is integrated into CabanaPD. Right: final damage field (yellow, orange, and red indicate high damage, and blue indicates no damage) simulated in CabanaPD as a result of vertical tensile loading. In this demonstration, the bonds between particles in the same grain are much stronger than the bonds between particles in different grains, which is why the damage follows the grain structure. A no-fail condition is enforced on the top and bottom planes where the force is applied, so no damage is visible in these regions.

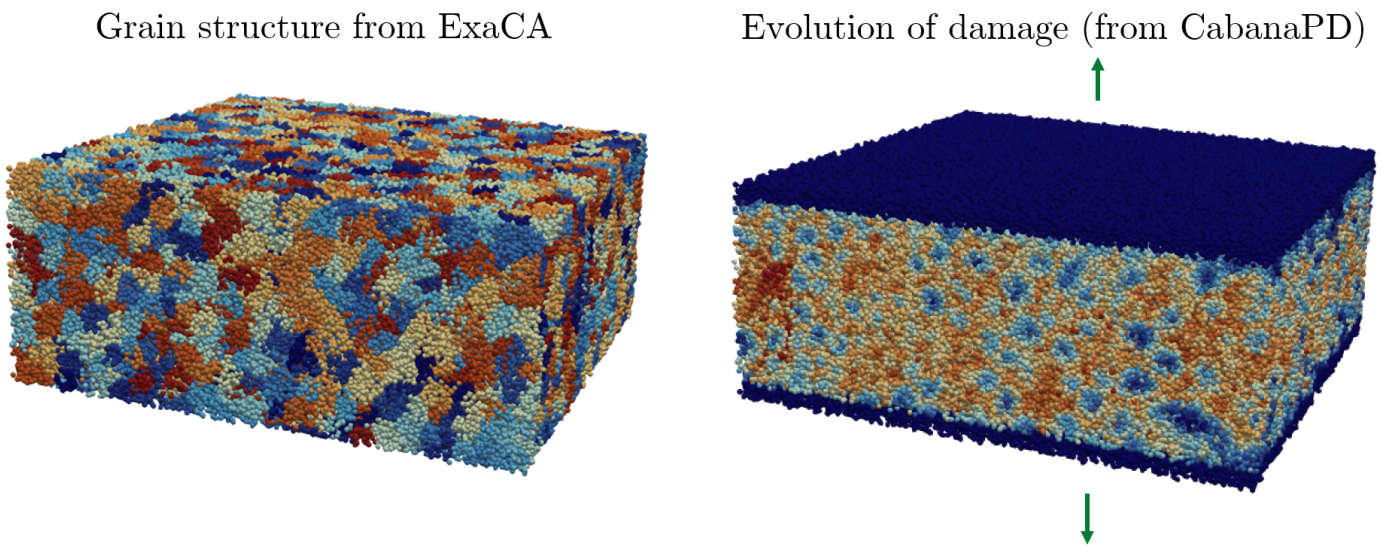


Figure 18: Left: example grain structure generated using ExaCA. Right: final damage field (yellow, orange, and red indicate high damage, and blue indicates no damage) simulated in CabanaPD as a result of vertical tensile loading. In this demonstration, the bonds between particles in the same grain are much stronger than the bonds between particles in different grains, which is why the damage follows the grain structure. A no-fail condition is enforced on the top and bottom planes where the force is applied, so no damage is visible in these regions.